Econ 97 Labs

Patrick Molligo

February 23, 2026

How to Use This Lab Manual

These labs are designed to reinforce *probability theory* by using **simulation** and **computational verification** in R.

Why simulation helps in probability:

- Many probabilities are defined as *long-run frequencies* (e.g., the chance of heads is the limiting proportion of heads in repeated coin flips).
- Simulation lets us **approximate** theoretical probabilities and see concepts like the **Law of Large Numbers** and **Central Limit Theorem** in action.
- Simulation is also a practical tool when exact calculations are difficult.

Throughout, we will use:

- **Random sampling** (`sample()`, `rbinom()`, etc.)
- **Indicators** like `(event)` which evaluate to `TRUE/FALSE`, and means of indicators to estimate probabilities
- **Repetition** (`replicate()`) to generate many trials
- **Visualization** (`plot()`, `hist()`) to connect distributions to data

---

Lab 1 — R Basics + Simulation + Law of Large Numbers

Objectives

- Learn R basics: vectors, arithmetic, logical comparisons, and summaries
- Estimate probabilities through simulation
- See the **Law of Large Numbers (LLN)**: the sample proportion approaches the true probability as sample size grows

Concepts

Probability as a long-run frequency

If we repeat an experiment many times, the proportion of times an event occurs tends to stabilize around the true probability. In simulation, we approximate a probability by:

$$P(A) \approx \frac{\text{trials where } A \text{ happens}}{\text{number of trials}}.$$

Indicator variables

If we define an indicator (I_A) that equals 1 when event (A) occurs and 0 otherwise, then:

- The average of (I_A) across many trials estimates (P(A)).

- In R, `(condition)` produces `TRUE/FALSE`, and `mean(TRUE/FALSE)` treats `TRUE` as 1 and `FALSE` as 0.

Warm-up (run each line)

```
1:10
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

```
sum(1:10)
```

```
## [1] 55
```

```
mean(1:10)
```

```
## [1] 5.5
```

```
c(2, 5, 9)^2
```

```
## [1]  4 25 81
```

Part A — Coin flips and LLN

Simulate coin flips and estimate (P(Heads)).

```
set.seed(1)
flips <- sample(c("H","T"), size = 50, replace = TRUE)
flips
```

```
##  [1] "H" "T" "H" "H" "T" "H" "H" "H" "T" "T" "H" "H" "H" "H" "H" "T" "T" "T" "T"
## [20] "H" "H" "H" "H" "H" "H" "H" "T" "H" "H" "T" "T" "T" "H" "T" "H" "H" "T" "H"
## [39] "T" "T" "T" "T" "H" "T" "T" "T" "T" "T" "H" "H"
```

```
mean(flips == "H")
```

```
## [1] 0.54
```

Repeat with larger sample sizes and observe stabilization.

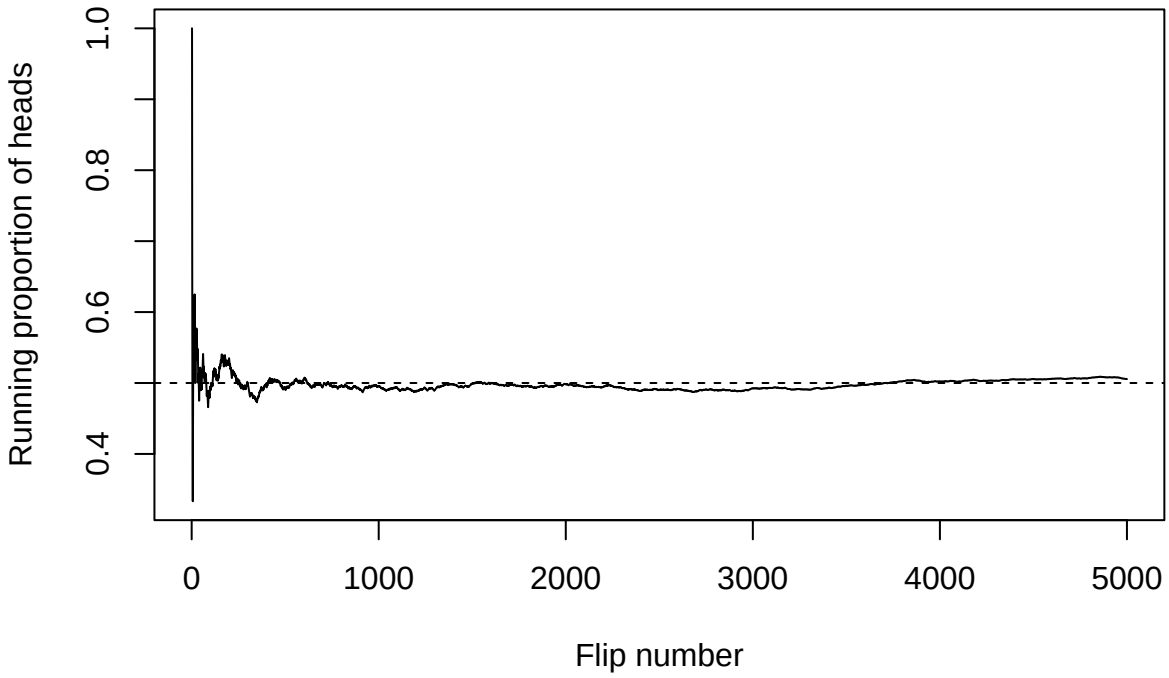
```
set.seed(1)
for (n in c(50, 500, 5000, 50000)) {
  flips <- sample(c("H","T"), size = n, replace = TRUE)
  cat("n =", n, " prop_heads =", mean(flips == "H"), "\n")
}
```

```
## n = 50  prop_heads = 0.54
## n = 500  prop_heads = 0.518
## n = 5000  prop_heads = 0.4902
## n = 50000  prop_heads = 0.5034
```

Running estimate (visual LLN)

A running estimate shows how the proportion evolves with each additional flip.

```
set.seed(2)
n <- 5000
x <- sample(c(1,0), size = n, replace = TRUE)  # 1=heads, 0=tails
running <- cumsum(x) / (1:n)
plot(running, type="l", xlab="Flip number", ylab="Running proportion of heads")
abline(h=0.5, lty=2)
```



Part B — Multiple-choice guessing

A 30-question quiz, 4 choices each. If you guess randomly, each question is correct with probability (p = 1/4). The total score is:

$$X \sim \text{Binomial}(n = 30, p = 1/4).$$

The theoretical expected score is ($E[X] = np = 30 \cdot 1/4 = 7.5$).

Simulate 10,000 students:

```
set.seed(3)
B <- 10000
scores <- replicate(B, sum(sample(1:4, 30, replace=TRUE) == 1))
mean(scores)
```

```
## [1] 7.4832
```

Estimate ($P(X \geq 12)$):

```
mean(scores >= 12)
```

```
## [1] 0.0542
```

Deliverables

- A short description (2–4 sentences) of what you see in the LLN running plot
- Estimated expected score and ($P(\text{score} \geq 12)$)

- Compare the simulated mean to the theoretical mean 7.5

Lab 2 — Counting + Sampling Without Replacement

Objectives

- Model draws **without replacement**
- Estimate conditional probabilities from sequential experiments
- Connect to **hypergeometric** reasoning and conditional probability rules

Concepts

Sampling without replacement

When items are not replaced, outcomes are **dependent**. For example, after drawing a sour milk, there are fewer sour milks remaining.

Conditional probability

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

In simulation, we estimate $(P(A \mid B))$ by restricting to the trials where (B) happened:

$$P(A \mid B) \approx \frac{\text{trials where } A \text{ and } B}{\text{trials where } B}.$$

Part A — One draw

Urn contains 7 good (G) and 3 sour (S).

```
urn <- c(rep("G", 7), rep("S", 3))
set.seed(10)
draw <- sample(urn, size=1, replace=FALSE)
draw
```

```
## [1] "S"
```

Simulate many single draws to estimate (P(S)).

```
set.seed(11)
B <- 10000
draws <- replicate(B, sample(urn, 1))
mean(draws == "S")
```

```
## [1] 0.2936
```

Part B — Sequential customers (conditional probability)

Simulate 3 sequential purchases (without replacement). Estimate:

$$P(\text{3rd is sour} \mid \text{2nd is not sour}).$$

```
set.seed(12)
B <- 20000

second_not_sour <- logical(B)
```

```r
third_sour <- logical(B)

for (b in 1:B) {
  purchase <- sample(urn, size=3, replace=FALSE)
  second_not_sour[b] <- (purchase[2] != "S")
  third_sour[b] <- (purchase[3] == "S")
}

mean(third_sour[second_not_sour])
```

```
## [1] 0.3299489
```

Deliverables

- Estimated ($P(S)$) for the first draw
- Estimated ($P(\text{3rd sour} \mid \text{2nd not sour})$)
- 2–3 sentences on why conditioning changes the probability

Lab 3 — Discrete Random Variables: Binomial + Visualization

Objectives

- Use `dbinom`, `pbinom`, `rbinom`
- Compare analytic probabilities to simulation
- Visualize a probability mass function (PMF)

Concepts

Binomial distribution

If (X) counts successes in (n) independent trials with success probability (p), then:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

In R:

- `dbinom(k, n, p)` gives ($P(X = k)$)
- `pbinom(k, n, p)` gives ($P(X \leq k)$)
- `rbinom(B, n, p)` simulates (B) values of (X)

Part A — Exact probabilities

Let ($X \sim \text{Binomial}(n = 20, p = 0.4)$).

```r
dbinom(8, size=20, prob=0.4)          # P(X=8)
```

```
## [1] 0.1797058
```

```r
pbinom(8, size=20, prob=0.4)          # P(X<=8)
```

```
## [1] 0.5955987
```

```r
pbinom(10, 20, 0.4) - pbinom(5, 20, 0.4)  # P(6<=X<=10)
```

```
## [1] 0.7468798
```

Part B — Simulation check

```
set.seed(20)
x <- rbinom(50000, size=20, prob=0.4)

mean(x == 8)
```

```
## [1] 0.17742
```

```
mean(x <= 8)
```

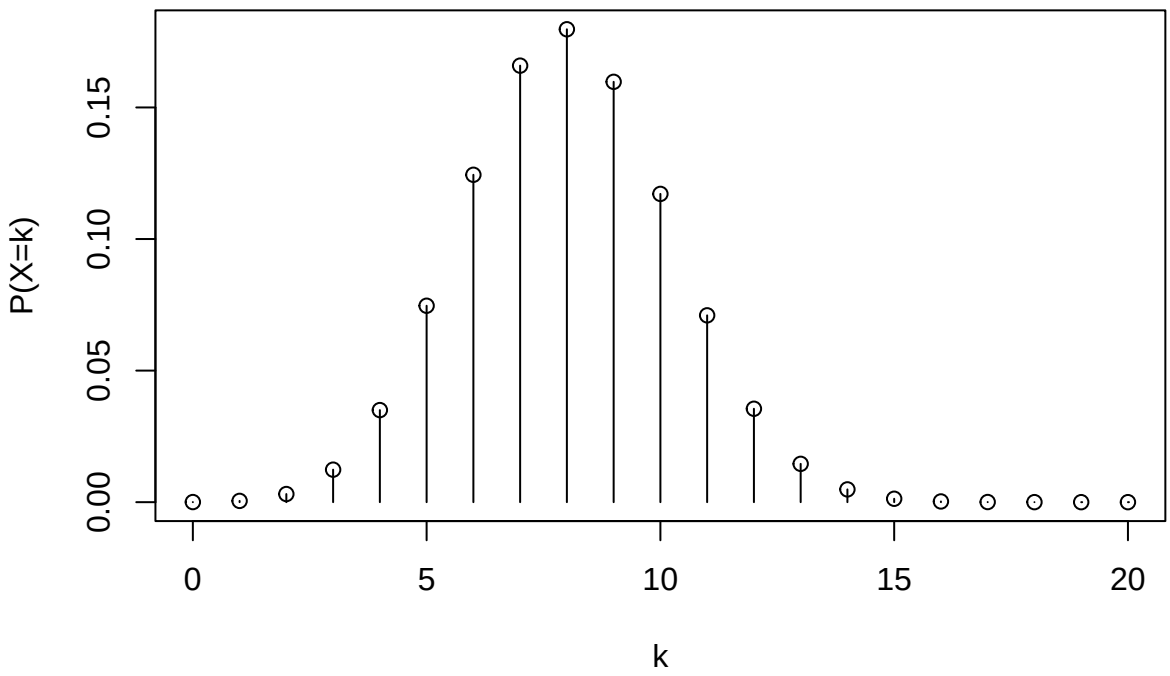
```
## [1] 0.59272
```

```
mean(x >= 6 & x <= 10)
```

```
## [1] 0.74414
```

Part C — Plot the PMF

```
k <- 0:20
pmf <- dbinom(k, 20, 0.4)
plot(k, pmf, type="h", xlab="k", ylab="P(X=k)")
points(k, pmf)
```



Deliverables

- Three probabilities (analytic + simulation)
- One paragraph explaining what a CDF is in words

Lab 4 — Geometric & Poisson + Approximations

Objectives

- Understand waiting times (geometric)
- Understand counts over intervals (Poisson)
- Compare analytic results to simulation

Concepts

Geometric distribution (waiting time)

If each trial succeeds with probability (p), and (T) is the number of trials until first success, then:

$$E[T] = \frac{1}{p}.$$

R note: `rgeom()` returns the number of failures before first success, so we add 1 to get trials-until-success.

Poisson distribution (counts)

If events happen randomly at average rate (λ) in an interval, then ($Y \sim \text{Poisson}(\lambda)$). In R:

- $dpois(y, lambda)$ gives ($P(Y = y)$)
- $ppois(y, lambda)$ gives ($P(Y \leq y)$)
- $rpois(B, lambda)$ simulates values

Part A — Geometric waiting time

```
set.seed(30)
p <- 0.2
T <- rgeom(20000, prob=p) + 1
mean(T)   # compare to 1/p = 5
```

```
## [1] 4.99595
```

```
mean(T > 8)
```

```
## [1] 0.16775
```

Part B — Poisson probabilities

Let ($Y \sim \text{Poisson}(\lambda = 3)$). Compute and simulate:

- ($P(Y = 0)$)
- ($P(Y \geq 5)$)

```
dpois(0, lambda=3)
```

```
## [1] 0.04978707
```

```
1 - ppois(4, lambda=3)
```

```
## [1] 0.1847368
```

```
set.seed(31)
Y <- rpois(50000, lambda=3)
mean(Y == 0)
```

```
## [1] 0.04894
```

```
mean(Y >= 5)
```

```
## [1] 0.1853
```

Deliverables

- Simulated vs theoretical (E[T])
- Two Poisson probabilities (analytic + simulation)

Lab 5 — Continuous Random Variables + Normal + Central Limit Theorem

Objectives

- Use `pnorm`, `rnorm`, `runif`
- Compute Normal probabilities
- See the **Central Limit Theorem (CLT)** using sample means

Concepts

Normal distribution and tail probabilities

For a standard normal ($Z \sim N(0,1)$):

- `pnorm(a)` computes ($P(Z \leq a)$)
- tail probabilities use `1 - pnorm(a)`

CLT (informal)

For many i.i.d. random variables with mean (μ) and standard deviation (σ), the sample mean ($\bar{X}$) becomes approximately Normal for large (n):

$$\bar{X} \approx N\left(\mu,\ \frac{\sigma}{\sqrt{n}}\right).$$

Part A — Normal probabilities

```
pnorm(1.25)                 # P(Z<1.25)
```

```
## [1] 0.8943502
```

```
2*(1 - pnorm(2))            # P(|Z|>2)
```

```
## [1] 0.04550026
```

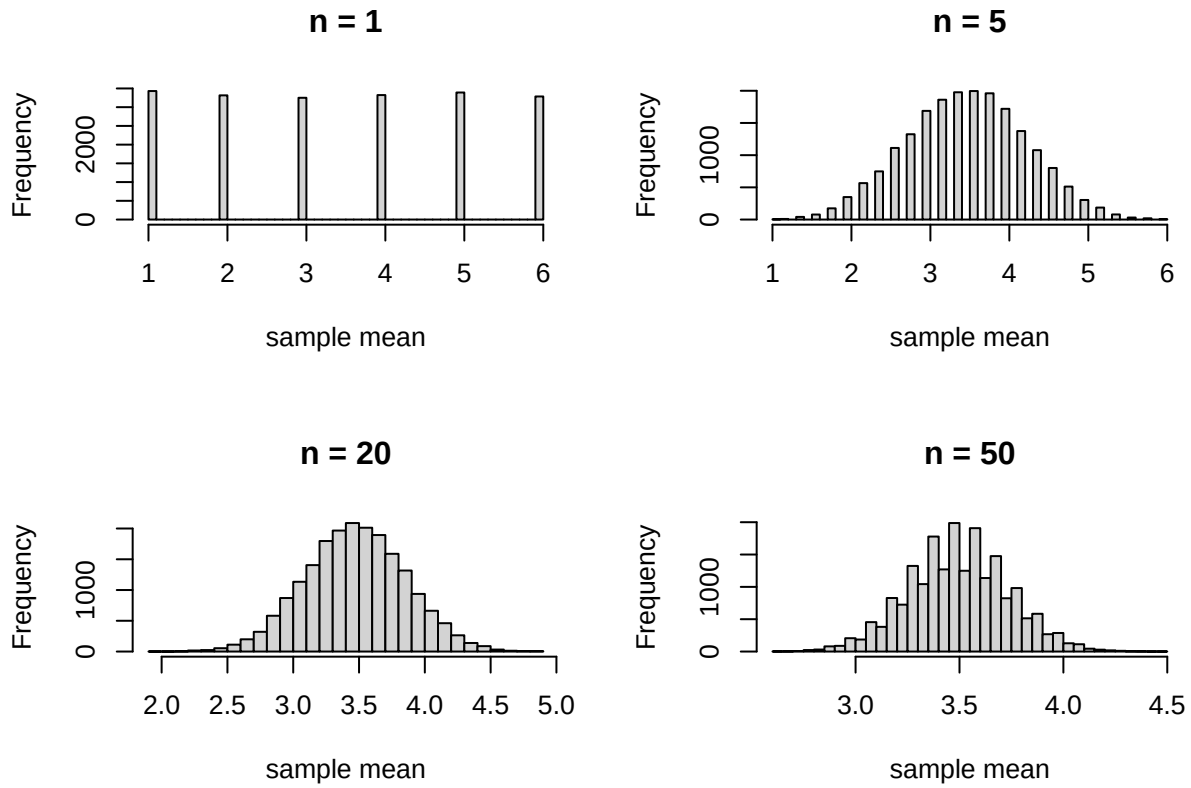
Part B — CLT with dice (distribution of sample means)

```
set.seed(40)
B <- 20000
ns <- c(1, 5, 20, 50)

means_list <- lapply(ns, function(n) {
  replicate(B, mean(sample(1:6, size=n, replace=TRUE)))
```

```
})

par(mfrow=c(2,2))
for (i in seq_along(ns)) {
  hist(means_list[[i]], main=paste("n =", ns[i]), xlab="sample mean", breaks=40)
}
```




```
par(mfrow=c(1,1))
```

Part C — Compare simulation to Normal approximation (optional)

For a fair die:

- $(\mu = 3.5)$
- $(\mathrm{Var}(X) = 35/12), so(\sigma = \sqrt{35/12})$

Compare $P(\bar{X} > 4)$ for (n=50).

```
n <- 50
xbar <- means_list[[which(ns==n)]]
p_sim <- mean(xbar > 4)

mu <- 3.5
sigma <- sqrt(35/12)
p_norm <- 1 - pnorm(4, mean=mu, sd=sigma/sqrt(n))

p_sim
```

```
## [1] 0.0176
```

```
p_norm
```

```
## [1] 0.01921697
```

Deliverables

- Two Normal probability answers
- The 2×2 histogram figure for CLT
- Simulation vs Normal approximation for (n=50)

Lab 6 — Estimation + Bootstrap Confidence Intervals

Objectives

- Understand sampling variability for an estimator (like the mean)
- Use the **bootstrap** to build a confidence interval

Concepts

Sampling variability

Even if the population is fixed, different samples produce different sample means.

Bootstrap idea

We approximate the sampling distribution of an estimator by:

1. Taking your observed sample
2. Resampling it *with replacement* many times (bootstrap samples)
3. Recomputing the estimator (mean) each time

A percentile bootstrap CI uses quantiles of bootstrap estimates.

Part A — Create a population and take a sample

We use an exponential distribution as a "population" because it is skewed (helpful for seeing why Normal approximations sometimes struggle with small samples).

```
set.seed(50)
pop <- rexp(200000, rate=1)  # true mean is 1
sample1 <- sample(pop, size=50, replace=FALSE)
mean(sample1)
```

```
## [1] 1.092376
```

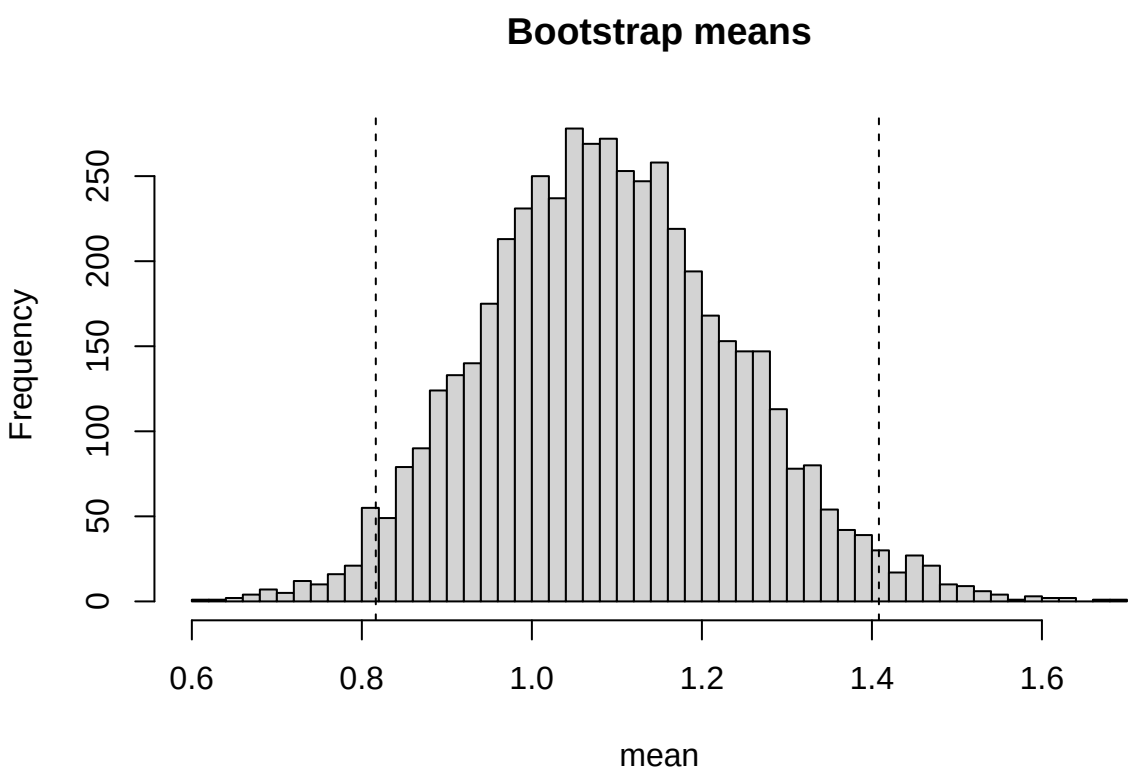
Part B — Bootstrap CI for the mean

```
set.seed(51)
B <- 5000
boot_means <- replicate(B, mean(sample(sample1, size=length(sample1), replace=TRUE)))

ci <- quantile(boot_means, c(0.025, 0.975))
ci
```

```
##      2.5%     97.5%
## 0.8164046 1.4082375
```

```r
hist(boot_means, breaks=40, main="Bootstrap means", xlab="mean")
abline(v=ci, lty=2)
```



Part C — Coverage experiment (optional challenge)

Repeat many times to estimate how often a 95% bootstrap CI contains the true mean 1.

```r
set.seed(52)
R <- 200      # number of repeated experiments
B <- 2000     # bootstrap reps per experiment

covers <- logical(R)

for (r in 1:R) {
  sample1 <- sample(pop, size=50, replace=FALSE)
  boot_means <- replicate(B, mean(sample(sample1, size=length(sample1), replace=TRUE)))
  ci <- quantile(boot_means, c(0.025, 0.975))
  covers[r] <- (ci[1] <= 1) & (1 <= ci[2])
}

mean(covers)
```

```
## [1] 0.88
```

Deliverables

- Your bootstrap 95% CI
- 3–5 sentences interpreting what the CI means (and what it does *not* mean)

Lab 7 — Hypothesis Testing via Simulation (p-values)

Objectives

- Interpret p-values as probabilities under the null model
- Compute p-values by exact calculation and by simulation
- Visualize the null distribution

Concepts

Null hypothesis and p-value

A hypothesis test begins with a null model (H_0). A p-value is:

> The probability, under (H_0), of seeing a result at least as extreme as the one observed.

This is a probability *about the data given the model*, not the probability that the model is true.

Scenario: Is a die biased toward 6?

Roll a die 60 times and observe 16 sixes.

Under (H_0): fair die, (p = 1/6), and

$$X \sim \text{Binomial}(n = 60, p = 1/6).$$

Part A — Exact one-sided p-value

Compute ($P(X \geq 16)$).

```
1 - pbinom(15, size=60, prob=1/6)
```

```
## [1] 0.0338462
```

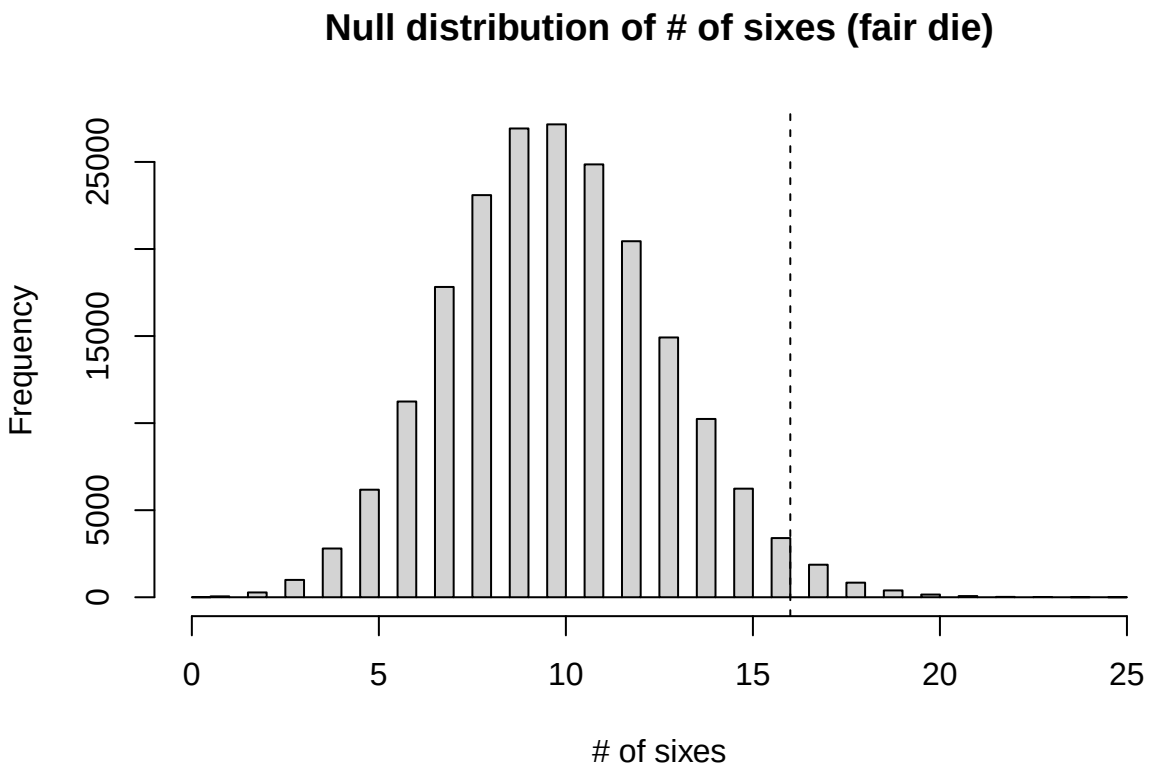
Part B — Simulation p-value

```
set.seed(60)
B <- 200000
X <- rbinom(B, size=60, prob=1/6)
p_sim <- mean(X >= 16)
p_sim
```

```
## [1] 0.03377
```

Part C — Visualize the null distribution

```
hist(X, breaks=50, main="Null distribution of # of sixes (fair die)", xlab="# of sixes")
abline(v=16, lty=2)
```



Deliverables

- Exact p-value and simulation p-value
- 3–5 sentences interpreting the p-value in plain English

General Rubric (apply to all labs)

- Code runs and is organized (30%)
- Numerical answers correct (30%)
- Written interpretation connects to probability concept (30%)
- Presentation: labeled axes, readable output, brief explanations (10%)